Lab 3: Component Modelling and Architectural Pattern Selection

Inter-City Freight Load Matching Marketplace

Name: Raksha V SRN: PES2UG24CS396 Section: G

Scenario Description

The Inter-City Freight Load Matching Marketplace is a digital platform that connects shippers who need to transport goods between cities with truck owners or drivers who have available vehicles. Shippers can post load details such as source, destination, load type, weight and delivery date. Truck owners can provide vehicle details, routes and availability. The system matches suitable trucks with freight loads and supports booking, payment, delivery tracking and notifications.

Functional Requirements

1. Users should be able to register and log in.
2. Shippers should be able to post freight-load details.
3. Truck owners should be able to register their vehicles.
4. Drivers should be able to update vehicle availability and routes.
5. The system should match suitable trucks with posted loads.
6. Shippers should be able to confirm bookings.
7. The system should support secure payments.
8. Drivers should be able to update delivery status.
9. Users should receive booking and delivery notifications.

Non-Functional Requirements

1. **Performance:** The system should return matching results quickly.
2. **Scalability:** It should support many shippers, drivers and bookings.
3. **Security:** User, shipment and payment information should be protected.
4. **Reliability:** Booking and payment information should be stored correctly.
5. **Availability:** The platform should be accessible whenever users need it.
6. **Usability:** The application should be easy for shippers and drivers to use.

Main Challenges

- Accurately matching loads with suitable trucks
- Processing many matching requests quickly
- Protecting user and payment information
- Preventing the same truck from being booked twice
- Maintaining correct booking and delivery information
- Supporting users operating in different cities

Architectural Style Analysis

A. Layered Architecture

In layered architecture, the system is divided into presentation, business-logic and data-access layers.

Advantages:

- Simple to understand and develop
- Clear separation between user interface, business logic and database

Disadvantages:

- Individual functions cannot be scaled easily
- Failure or heavy load in one layer may affect the entire application

B. Client-Server Architecture

In client-server architecture, shippers and drivers use client applications that communicate with a centralized server.

Advantages:

- Centralized management of users and freight data
- Easy to maintain consistent booking information

Disadvantages:

- The central server can become a performance bottleneck
- Server failure may make the complete marketplace unavailable

C. Microservices Architecture

In microservices architecture, the system is divided into independent services such as user management, load management, matching, booking and payment.

Advantages:

- Matching, booking and payment services can be scaled independently
- Failure in one service does not necessarily stop the entire system
- Services can be developed and updated separately

Disadvantages:

- Communication between services is more complex
- Maintaining data consistency between booking and payment services is challenging
- Deployment and monitoring require additional effort

6. Architecture Selection

We selected Microservices Architecture for the Inter-City Freight Load Matching Marketplace.

Reason 1: The number of load and truck-matching requests may increase significantly. The Matching Service can be scaled independently without scaling the entire application.

Reason 2: Functions such as user management, load posting, matching, booking, payment and notifications can be developed and maintained as separate services.

Security advantage: The Payment Service can be isolated from other components and accessed through authenticated and encrypted APIs, reducing exposure of sensitive payment information.

Performance benefit: Frequently used services such as Load Management and Matching can use independent resources and databases, producing faster search and matching results.

Architecture Selection and Component Identification

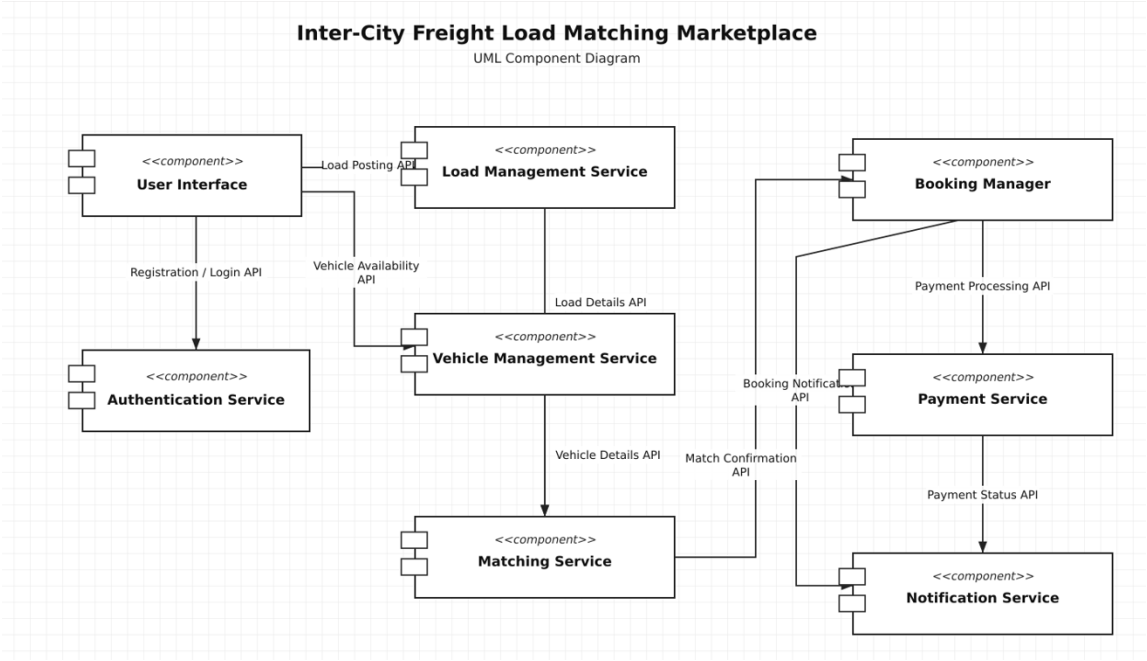
Selected Architecture

We selected **Microservices Architecture** for the Inter-City Freight Load Matching Marketplace.

Identified Components

1. **User Interface Component** – Provides the interface for shippers and truck owners.
2. **Authentication Service** – Handles registration, login and user authorization.
3. **Load Management Service** – Manages freight-load postings and load details.
4. **Vehicle Management Service** – Manages truck details, capacity, routes and availability.
5. **Matching Service** – Matches freight loads with suitable available trucks.
6. **Booking Manager Component** – Creates and manages confirmed freight bookings.
7. **Payment Service Component** – Processes payments and maintains payment status.
8. **Notification Service** – Sends booking, payment and delivery notifications.

Component Diagram



Architecture Justification

Scenario: Inter-City Freight Load Matching Marketplace
Selected Architecture: Microservices Architecture

Architectural Choice

We chose **Microservices Architecture** for the Inter-City Freight Load Matching Marketplace. In this architecture, the system is divided into independent components such as Authentication Service, Load Management Service, Vehicle Management Service, Matching Service, Booking Manager, Payment Service and Notification Service. These components communicate with each other through APIs.

Reasons for Selection

1. Independent scalability:
The marketplace may receive many freight postings and truck-matching requests from different cities. Microservices architecture allows frequently used components, especially the Load Management Service and Matching Service, to be scaled independently without scaling the entire application.

2. Fault isolation and maintainability:
Each service performs a specific function and can be developed, tested and updated separately. If the Notification Service temporarily fails, important services such as booking, matching and payment can continue operating.

Security Advantage

Sensitive payment operations are isolated within the Payment Service. Only authenticated and authorized requests can access this service through secure and encrypted APIs. This reduces the exposure of users' personal and payment information.

Performance Benefit

The Matching Service can use dedicated computing resources to compare freight requirements with vehicle capacity, route and availability. Independent scaling and database optimization allow the system to provide faster matching results even when the number of users increases.

Conclusion

Microservices architecture is suitable for the Inter-City Freight Load Matching Marketplace because it provides scalability, fault isolation, security, flexibility and better performance. It also enables the major services of the marketplace to be maintained and improved independently.